

How to Sort a List of Complex Types in C#

In this article, I am going to discuss **How to Sort a List of Complex Types in C#** like Employee, Customer, Product, Department, etc. with examples. Please read our previous article where we discussed [Generic List in C#](#) with Examples. Before understanding, how to sort a complex type, let us first understand how to sort simple types like int, double, char, string, etc.

Generic List<T> Collection Class Sort Methods in C#:

The Generic List Collection Class in C# provides the following four Sort Methods.

1. **Sort()**: This method is used to sort the elements in the entire Generic List using the default comparer.
2. **Sort(IComparer<T> comparer)**: This method is used to sort the elements in the entire Generic List using the specified comparer.
3. **Sort(Comparison<T> comparison)**: This method is used to sort the elements in the entire Generic List using the specified System.Comparison.
4. **Sort(int index, int count, IComparer<T> comparer)**: This method is used to sort the elements in a range of elements in a Generic List using the specified comparer.

How to Sort a List of Simple Types in C#:

Sorting a generic list collection of simple types is very straightforward. We just need to invoke the Sort() method (Provided by the Generic List class) on the list instance and the data will be automatically sorted in ascending order. For example, if we have a list of integers as shown below

```
List<int> numbersList = new List<int>{ 1, 8, 7, 5, 2 };
```

Then we just need to invoke the Sort() method on numbersList collection as shown below

```
numbersList.Sort();
```

If you want the data to be retrieved in descending order, then after calling the Sort method, you need to call the Reverse() method on the list instance as shown below.

```
numbersList.Reverse();
```

Example to Understand How to Sort a List Collection of Simple Types in C#:

In the below example, we are creating list collections of int and string types and then calling the Sort Method to Sort the collection and the Reverse Method to reverse the collection.

```
using System;
using System.Collections.Generic;
namespace ListCollectionSortReverseMethodDemo
{
    public class Program
    {
        public static void Main()
        {
            List<int> numbersList = new List<int> { 1, 8, 7, 5, 2 };
            Console.WriteLine("Numbers Before Sorting");
```

```

foreach (int i in numbersList)
{
    Console.Write($"{i} ");
}
// The Sort() of List Collection class will sort the data in ascer
numbersList.Sort();
Console.WriteLine("\n\nNumbers After Sorting");
foreach (int i in numbersList)
{
    Console.Write($"{i} ");
}
// If you want to retrieve data in descending order then use the f
method
numbersList.Reverse();
Console.WriteLine("\n\nNumbers in Descending order");
foreach (int i in numbersList)
{
    Console.Write($"{i} ");
}
//Another Example of Sorting String
List<string> names = new List<string>() { "Pranaya", "Anurag", "Sa
"Hina", "Rakesh" };
Console.WriteLine("\n\nNames Before Sorting");
foreach (string name in names)
{
    Console.WriteLine(name);
}
names.Sort();
Console.WriteLine("\nNames After Sorting");
foreach (string name in names)
{
    Console.WriteLine(name);
}
names.Reverse();
Console.WriteLine("\nNames in Descending Order");
foreach (string name in names)
{
    Console.WriteLine(name);
}
Console.ReadKey();
}
}
}

```

Output:

```
Numbers Before Sorting
1 8 7 5 2

Numbers After Sorting
1 2 5 7 8

Numbers in Descending order
8 7 5 2 1

Names Before Sorting
Pranaya
Anurag
Sambit
Hina
Rakesh

Names After Sorting
Anurag
Hina
Pranaya
Rakesh
Sambit

Names in Descending Order
Sambit
Rakesh
Pranaya
Hina
Anurag
```

However, when we do the same thing on a complex type like Employee, Product, Customer, Department, etc. we will get a runtime exception as “**Invalid operation exception – Failed to compare 2 elements in the array**”. For a better understanding, please have a look at the below example. Here, we are creating a collection of Employees and then calling the Sort method which will throw a runtime exception.

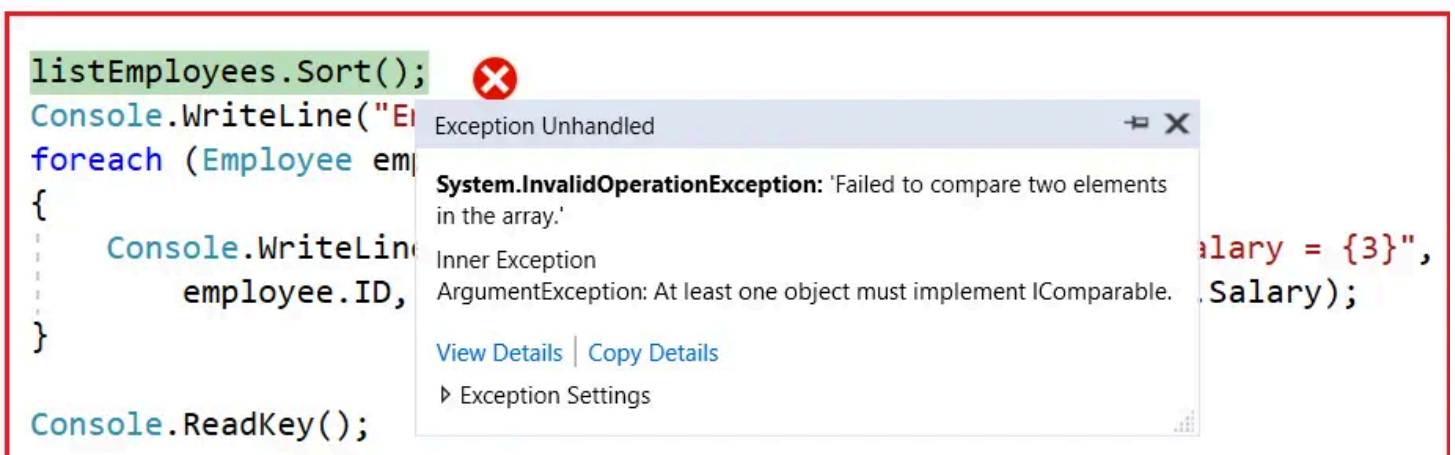
```
using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male",
5000 },
```

```

    new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
    new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
    new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
    new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
};
Console.WriteLine("Employees Before Sorting");
foreach (Employee employee in listEmployees)
{
    Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary = {3}",
        employee.ID, employee.Name, employee.Gender, employee.Salary);
}
listEmployees.Sort();
Console.WriteLine("\nEmployees After Sorting");
foreach (Employee employee in listEmployees)
{
    Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary = {3}",
        employee.ID, employee.Name, employee.Gender, employee.Salary);
}
Console.ReadKey();
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:



We are getting the above Runtime Exception because at runtime the .NET Framework does not identify how to sort the complex types. In this, the .NET Runtime is unable to identify whether to sort the data based on ID property, or based on Name property, or based on Gender property, or based on Salary property, or a combination of any properties. So, if we want to sort a complex type then we need to tell the way we want the data to be sorted in the list, and to do this we need to implement the IComparable interface.

How the sort functionality is working for simple data types like int, double, string, char, etc. in C#?

This is working because these types (int, double, string, decimal, char, etc.) are already implementing the IComparable interface. If you go to the definition of any built-in types, then you will see that the class is implemented IComparable interface as shown in the below image and this is the reason why the sort functionality is working.

```
...public struct Int32 : IComparable, IFormattable, IConvertible, IComparable<Int32>, IEquatable<Int32>
{
    ...public const Int32 MaxValue = 2147483647;
    ...public const Int32 MinValue = -2147483648;

    ...public sealed class String : IComparable, ICloneable, IConvertible, IEnumerable,
    {
        ...public static readonly String Empty;
```

How to Sort a List of Complex Types in C#?

To sort a list of complex types without using LINQ, the complex type has to implement the **IComparable** interface and needs to provide the implementation for the **CompareTo()** method as follows. The **CompareTo()** method returns an integer value and the meaning of the return value as shown below.

1. **Return value greater than ZERO** – The current instance is greater than the object being compared with.
2. **Return value less than ZERO** – The current instance is less than the object being compared with.
3. **The Return value is ZERO** – The current instance is equal to the object being compared with.

Alternatively, we can also invoke the CompareTo() method directly. The Salary property of the Employee object is int and the CompareTo() method is already implemented on the integer type that we already discussed, so we can invoke this method and return its value as shown below.

```
return this.Salary.CompareTo(obj.Salary);
```

Implementing the IComparable interface in C# in Employee Class

Let us understand this with an example. What we want is, we need to sort the employees based on their Salary. To do so, our Employee class should implement the **IComparable** interface and should provide an implementation for the **CompareTo()** method. This method will compare the current object (specified with this) and the object to be compared which it will get as a parameter. So, modify the Employee class as follows and the following code exactly does the same.

```

public class Employee : IComparable<Employee>
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
    public int CompareTo(Employee obj)
    {
        if (this.Salary > obj.Salary)
        {
            return 1;
        }
        else if (this.Salary < obj.Salary)
        {
            return -1;
        }
        else
        {
            return 0;
        }
    }
}

```

The following is the complete example code.

```

using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", 5000 },
                new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", 7000 },
                new Employee() { ID = 103, Name = "Anurag", Gender = "Male", 5500 },
                new Employee() { ID = 104, Name = "Sambit", Gender = "Male", 6500 },
                new Employee() { ID = 105, Name = "Hina", Gender = "Female", 6500 }
            };
            Console.WriteLine("Employees Before Sorting");
            foreach (Employee employee in listEmployees)
            {

```

```

        Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary = {3}",
            employee.ID, employee.Name, employee.Gender, employee.Salary);
    }

    listEmployees.Sort();
    Console.WriteLine("\nEmployees After Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary = {3}",
            employee.ID, employee.Name, employee.Gender, employee.Salary);
    }
    Console.ReadKey();
}

public class Employee : IComparable<Employee>
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
    public int CompareTo(Employee obj)
    {
        if (this.Salary > obj.Salary)
        {
            return 1;
        }
        else if (this.Salary < obj.Salary)
        {
            return -1;
        }
        else
        {
            return 0;
        }
    }
}
}

```

Now run the above code and you will get the result in ascending order based on the Employee Salary as shown in the below image.

```

Employees Before Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500

Employees After Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000

```

If you do not prefer to use the Sort functionality provided by the Employee class, then you can provide your own implementation by implementing the **IComparer** interface. For example, if you want the employees to be sorted by **Name** instead of **Salary** then you need to follow the below two steps.

Step1: Implement the IComparer interface

Create a class implementing the **IComparer<T>** interface and provide implementation to the Compare method as follows.

```

public class SortByName : IComparer<Employee>
{
    public int Compare(Employee x, Employee y)
    {
        return x.Name.CompareTo(y.Name);
    }
}

```

Step2: Use the overloaded version of the Sort method which takes the IComparer interface as a parameter:

Pass an instance of the class that implements the IComparer interface as an argument to the Sort() method as shown below.

```

SortByName sortByName = new SortByName();
listEmployees.Sort(sortByName);

```

Complete Example to use own Comparer to Sort Employee by Name:

```

using System;
using System.Collections.Generic;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()

```

```

{
    //Creating a list of type Employee
    List<Employee> listEmployees = new List<Employee>
    {
        new Employee() { ID = 101, Name = "Pranaya", Gender = "Male",
5000 },
        new Employee() { ID = 102, Name = "Priyanka", Gender = "Female",
7000 },
        new Employee() { ID = 103, Name = "Anurag", Gender = "Male",
5500 },
        new Employee() { ID = 104, Name = "Sambit", Gender = "Male",
6500 },
        new Employee() { ID = 105, Name = "Hina", Gender = "Female",
6500 }
    };
    Console.WriteLine("Employees Before Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary
            employee.ID, employee.Name, employee.Gender, employee.Sal
        }
    }
    SortByName sortByName = new SortByName();
    listEmployees.Sort(sortByName);
    Console.WriteLine("\nEmployees After Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine("ID = {0}, Name = {1}, Gender = {2}, Salary
            employee.ID, employee.Name, employee.Gender, employee.Sal
        }
    }
    Console.ReadKey();
}
}

public class SortByName : IComparer<Employee>
{
    public int Compare(Employee x, Employee y)
    {
        return x.Name.CompareTo(y.Name);
    }
}

public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

Employees Before Sorting

```
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
```

Employees After Sorting

```
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
```

Sorting Collection of Complex Types using LINQ in C#:

Using LINQ `OrderBy` and `OrderByDescending` methods we can easily sort the elements of a collection of complex types in C#. What we need to do is, we need to call the `OrderBy` method if we want to Sort the elements in ascending and the `OrderByDescending` method if we want to sort the elements in descending order. For a better understanding please have a look at the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace ListCollectionDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a list of type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 },
                new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
                new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
                new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
                new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
            };
            Console.WriteLine("Employees Before Sorting");
            foreach (Employee employee in listEmployees)
            {
                Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}, Gender = {employee.Gender}, Salary = {employee.Salary}");
            }
        }
    }
}
```

```

    }
    //Sorting Employee using Linq OrderBy Method
    listEmployees = listEmployees.OrderBy(x => x.Name).ToList();
    Console.WriteLine("\nEmployees After Sorting using Linq");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    Console.ReadKey();
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

```

Employees Before Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500

Employees After Sorting using Linq
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500

```

In the next article, I am going to discuss the [Comparison Delegate in C#](#) with Examples. Here, in this article, I try to explain **How to Sort a List of Complex Types in C#** with Examples. I hope this How to Sort a List of Complex Types in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.